

Startersjabloon

Dit is een leeg vertrekpunt. De **projectstructuur maak je zelf** — dat is onderdeel van de opdracht, en het is een examenvraag: *hoe ga je om met modules in je project setup?*

Versies: Spring Boot 4.0.8 • Java 21 Inleveren: je eigen GitHub-repository

Beoordeling: mondelinge verdediging

01 Wat je in week 1 opzet

Een Maven multi-module project met vier services:

SERVICE	POORT
employee-service	8081
department-service	8082
organization-service	8083
notification-service	8084

1. Nieuw Maven-project zonder archetype — dat wordt je **root**. Zet daar `<packaging>pom</packaging>` en voeg de vier modules toe.
2. Per service: *New Module* → *Spring Initializr* met Web, Data JPA, Validation en MySQL Driver.
3. Controleer dat `mvn clean package` in de root alle vier de modules bouwt.

02 Kies expliciet deze versies

```
Spring Boot 4.0.8
Java        21
```

De wizard biedt standaard de nieuwste Spring Boot aan. Die hoort **niet** bij Spring Cloud 2025.1, dat je vanaf H4 nodig hebt. Zet hem dus handmatig terug.

Spring Cloud voeg je pas toe vanaf H4 — in week 2 en 3 werk je met gewone Spring Boot.

03 Het domeinmodel — wat vastligt

Dit ligt vast, de rest bepaal je zelf:

ENTITY	VERPLICHTE VELDEN
Employee	firstName, lastName, email, departmentId, organizationId
Department	name, organizationId
Organization	name

De vetgedrukte verwijsvelden zijn niet optioneel: zonder die kan je vanaf H4 geen enkele relatie leggen tussen je services. Let op dat het gewone Long -velden zijn en geen @ManyToOne naar een entity uit een andere service — die bestaat niet in jouw database.

notification-service heeft geen entity; die bewaart niets.

Wat je er verder bij zet is vrij: adres, geboortedatum, functie. Doe dat gerust — een model met alleen de verplichte velden is saai om over te praten op je verdediging.

04 Eén regel die je hele semester scheelt

Zet in je root pom geen enkele <dependency> — alleen <dependencyManagement>. Wat een module nodig heeft, zet je in die module. Waarom dat ertoe doet, zie je in H2 en opnieuw in H5, wanneer je config-service anders probeert aan te melden bij een discovery-service die nog niet draait.

05 Elke database apart

Elke service krijgt een eigen schema — geen gedeelde tabellen. Twee services aan dezelfde tabel kunnen niet meer onafhankelijk wijzigen of deployen, en dan heb je de koppeling terug die je net wilde wegnemen.

06 Wat je per week bouwt

WEEK	WAT ERBIJ KOMT
H2	De vier services met CRUD. Nog geen Spring Cloud.
H3	Tests: Mockito, MockMVC, Testcontainers
H4	Discovery service en Open Feign
H5	Config service en API gateway
H6	Centrale logging en tracing
H7	Messaging met AMQP, en resilience
H8	Dockerize en API-documentatie

Wat je bouwt blijft staan — er komt telkens alleen iets bij.

Per hoofdstuk staat op de slides **wanneer je klaar bent**. Loop die criteria af voor je aan het volgende hoofdstuk begint.

07 Wat er per week in je repository staat

Aan het eind van elke week draait een script over alle repositories. Het leest geen code en beoordeelt niets — het kijkt alleen of er gecommit is en of de dingen hieronder er staan. Het resultaat krijg je als **GitHub Issue** in je eigen repository: wat er staat, wat er nog mist, en wat de volgende week vraagt. Het telt niet mee; het is bedoeld om op tijd bij te sturen. Klopt er iets niet — heb je het anders genoemd of ergens anders gezet — antwoord dan gewoon op het issue.

WEEK	AANWEZIG IN JE REPOSITORY
1	Root pom.xml met <packaging>pom</packaging> en vier <module> s, géén <dependencies> in de root; Spring Boot 4.0.8; Java 21
2	Vier servicemappen; Employee met departmentId en organizationId, Department met organizationId, Organization; geen @ManyToOne naar een andere service; @RestController en @Valid in je services; elke service een eigen datasource
3	Testklassen per service; ArgumentCaptor; MockMvc; verify(..., never()); Testcontainers in je pom; nergens @Disabled of @Ignore
4	spring-cloud-dependencies 2025.1 in je root pom; een discovery-service; Eureka-client in elke service; @FeignClient zonder localhost
5	Een config-service; spring.config.import in elke service; een gateway; CORS op de gateway
6	micrometer-tracing en feign-micrometer; Loki; AMQP in je pom; employee.created; een @RabbitListener
7	Resilience4j met een fallback; een Dockerfile per service; docker-compose.yml zonder localhost; springdoc-openapi

Wie in week 4 nog geen @FeignClient heeft, loopt achter — en dan kom ik langs bij de start van het lab. Dat is geen controle, dat is hulp op tijd.

Commit **regelmatig**, niet één keer op het eind. Je commits laten zien hoe je project gegroeid is, en dat is bruikbaar materiaal op je verdediging. Een repository waar drie weken niets in gebeurt, valt op.

Dit is de README van het startersjabloon, zoals je ze in week 1 kloont. Verandert er iets, dan staat de laatste versie in het jabloon zelf.